Project 3: Distributional Semantics and Text Classification on a Kazakh News Corpus

Bahruz Gurbanli Madina Kylyshkanova

ADA University
CSCI 6515: Natural Language Processing

Team Contribution

Team Member	Contribution
Bahruz Gurbanli	Participated in experimental design, result analysis, report preparation, and presentation preparation. Primary implementation responsibility: Tasks 4–6, including embedding comparison, text classification, repository integration, and the interactive UI.
Madina Kylyshkanova	Participated in experimental design, result analysis, report preparation, and presentation preparation. Primary implementation responsibility: Tasks 1–3, including dataset description, matrix construction, Word2Vec, and GloVe experimentation.

Project repository placeholder: https://github.com/behruzgurbanli/nlp_corpora_and_algorithms_in_kazakh_language

1 Motivation

This project extends the cleaned Kazakh (kk-Latn) news corpus used in Projects 1 and 2 toward distributional semantics and neural text classification. The work addresses three connected questions. First, what lexical and co-occurrence structure is present in the corpus? Second, how well do Word2Vec and GloVe capture semantic similarity on this data? Third, which feature spaces are most effective for text classification when paired with recurrent models?

The project was implemented as a continuation of the same shared repository rather than as a separate prototype. The Project 3 code was integrated into the existing CLI and configuration structure, and the final system also includes a Streamlit UI for interactive demonstration.

2 Objectives

The project objectives were:

- to describe the cleaned Kazakh news corpus through corpus size, vocabulary size, word frequency, and matrix-based lexical structure;
- to train and evaluate Word2Vec and GloVe embeddings on the same dataset and compare their qualitative behavior;

- to compare classical sparse features and embedding-based features for document classification using the assignment-required neural architectures;
- to integrate all Project 3 tasks into the same reproducible repository structure used in the earlier project stages.

3 Method

3.1 Dataset and Task 1

The experiments use `data/processed/qz_kazakh_latn_clean_norm.jsonl` with the field `clean_text`. The current corpus statistics are shown in Table 1.

Table 1: Corpus statistics

Statistic	**Value**
Documents	2473
Total words	532829
Distinct words	55473
Rare words with frequency 1	27152
Words with frequency 2	8325
Frequent words with frequency at least 3	19996

Let the vocabulary be $V = \{w_1, \ldots, w_m\}$ and the document set be $D = \{d_1, \ldots, d_n\}$. If $c(w_i, d_j)$ denotes the count of word w_i in document d_j, then the term-document matrix is

$$T_{ij} = c(w_i, d_j).$$

In our implementation, the word-word matrix is defined through document-level co-occurrence:

$$W_{ij} = \sum_{d \in D} \mathbf{1}[w_i \in d \wedge w_j \in d,\ i \neq j].$$

Therefore, two selected words are treated as co-occurring if they appear in the same document. Since the full matrices are too large to visualize clearly, we restrict the figures to the top 20 words and the first 50 documents. This preserves interpretability without changing the underlying dataset statistics.

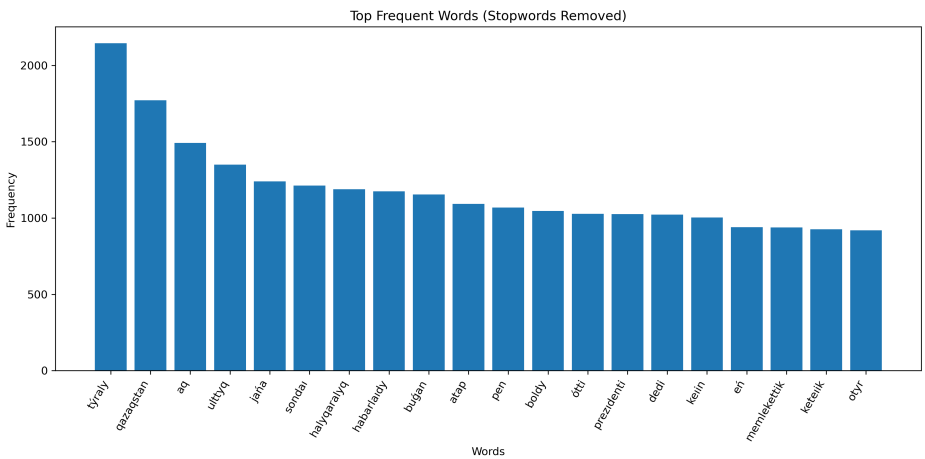

Figure 1: Top 20 frequent words after stopword removal.

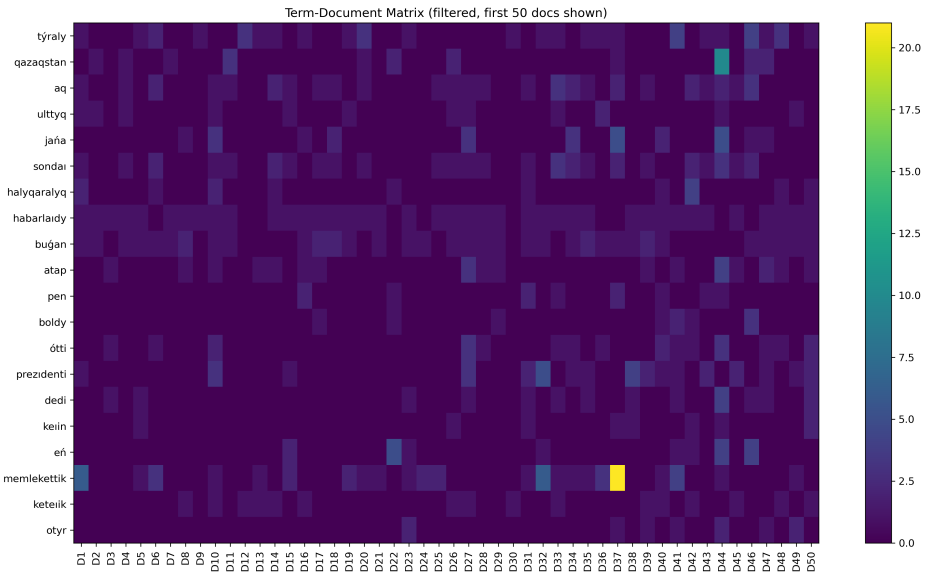

Figure 2: Filtered term-document matrix for the top 20 words and the first 50 documents.

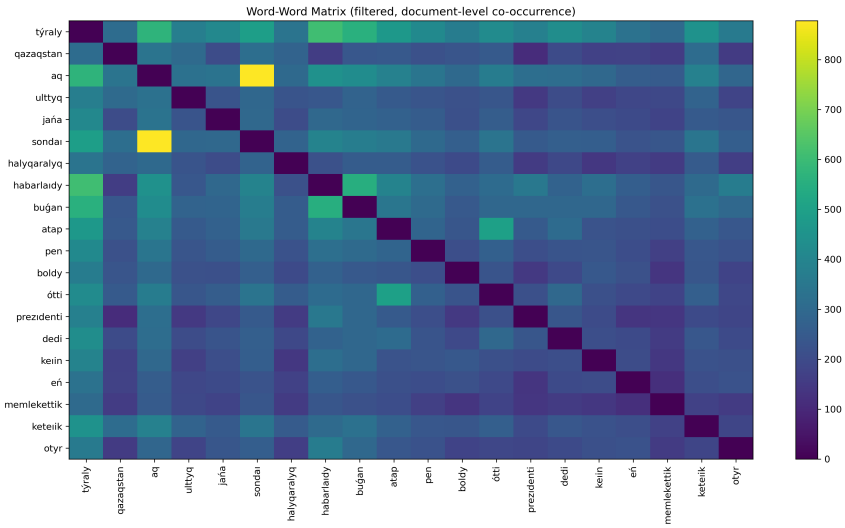

Figure 3: Filtered word-word matrix based on document-level co-occurrence.

3.2 Task 2: Word2Vec

Word2Vec was trained on sentence-level token sequences extracted from the cleaned corpus. We used the skip-gram architecture with negative sampling. For a center word w_t and a context word w_c, the objective is

$$\log \sigma({v'_{w_c}}^{\top} v_{w_t}) \; + \; \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \log \sigma(-{v'_{w_i}}^{\top} v_{w_t}),$$

where k is the number of negative samples and $P_n(w)$ is the noise distribution.

The selected parameters were vector size 100, window size 5, minimum count 3, negative samples 10, and 15 training epochs. The final training set contained 39,809 sequences and produced a vocabulary of 24,481 tokens. We evaluated the model on 10 target words using nearest neighbors and vector arithmetic. Representative outputs were:

- `tramp` → `tramptyń`, `baıden`, `prezıdent_donald`
- `aqsh` → `vashıngton`, `marko_rýbıo`, `vens`
- `jumys` → `isteýin`, `isteýine`, `isteýde`

These results show that Word2Vec learned useful local-context relations, especially for frequent political and institutional vocabulary. At the same time, the neighbors should be interpreted as contextually similar words rather than strict dictionary synonyms.

Table 2: Selected Word2Vec pairwise cosine similarities

Word 1	**Word 2**	**Similarity**
`tramp`	`donald`	0.3530
`aqsh`	`tramp`	0.2754
`jumys`	`isteý`	0.3822
`qazaqstan`	`prezıdenti`	0.1216
`ulttyq`	`halyqaralyq`	0.0473

3.3 Task 3: GloVe

GloVe was trained from global co-occurrence counts. Its objective is

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2,$$

where X_{ij} is the co-occurrence count, w_i and $\tilde{w}_j$ are the word and context vectors, and $f(\cdot)$ is the weighting function.

The selected parameters were vector size 100, window size 5, minimum count 3, maximum iterations 15, and $x_{\max} = 100$. Training used 532,829 tokens and produced a vocabulary of 19,997 tokens. The model returned several meaningful clusters, for example:

- `jumys` → `isteıtin`, `istegen`, `isteýge`
- `tramp` → `donald`, `tramptyń`, `ákimshiligi`

However, compared with Word2Vec, GloVe outputs were generally noisier on this corpus. For example, the query `aqsh` also returned fragments such as `ta` and `tyń`. This suggests that, in our setting, the global co-occurrence approach was more sensitive to frequency artifacts.

Table 3: Selected GloVe pairwise cosine similarities

Word 1	**Word 2**	**Similarity**
`tramp`	`donald`	0.9737
`aqsh`	`tramp`	0.8697
`jumys`	`isteý`	0.5490
`qazaqstan`	`prezıdenti`	0.5735
`ulttyq`	`halyqaralyq`	0.7168

3.4 Task 4: Word2Vec vs. GloVe

To compare the two embedding families directly, we measured overlap between their top-k neighbor sets for the same query word. If $N_k^{(w2v)}(q)$ and $N_k^{(glove)}(q)$ denote the Word2Vec and

GloVe neighbor sets for query q, then Jaccard overlap is

$$J(q) = \frac{\left|N_k^{(w2v)}(q) \cap N_k^{(glove)}(q)\right|}{\left|N_k^{(w2v)}(q) \cup N_k^{(glove)}(q)\right|}.$$

Across 10 query words, the average overlap was 0.0111. This is extremely low and indicates that the two models capture different corpus regularities. In practice, Word2Vec emphasized predictive local context, while GloVe emphasized broader corpus-level co-occurrence structure.

Table 4: Embedding comparison summary

Measure	**Value**
Query words compared	10
Average Jaccard overlap	0.0111

3.5 Task 5: Text Classification

The current saved experiment uses the harder `subcategory` setting with eight labels: `Amerıka`, `Ekonomika`, `Eýrazııa`, `Halyqaralyq qatynastar`, `Mádenıet`, `Ortalyq Azııa`, `Prezıdent`, and `Sport`. The train/dev/test split is 1978/247/248. The class distribution is imbalanced: `Ortalyq Azııa` has 523 documents, several classes have 450 documents, and three classes have only 50 documents each. For that reason, macro F1 is more informative than raw accuracy.

We compared five feature families:

- Count Vectorizer
- TF-IDF
- PMI-derived dense vectors
- averaged Word2Vec document vectors
- averaged GloVe document vectors

PMI and PPMI were defined as

$$\mathrm{PMI}(w,c) = \log \frac{P(w,c)}{P(w)P(c)}, \qquad \mathrm{PPMI}(w,c) = \max(\mathrm{PMI}(w,c), 0).$$

After constructing the PPMI matrix, we applied truncated SVD to obtain dense representations.

The assignment-required neural models were RNN, BiRNN, and LSTM. We also included Logistic Regression as the required baseline. The recurrent models follow the standard updates

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

for a standard RNN, and

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f),$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o), \quad \tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c),$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t)$$

for LSTM.

To keep the comparison uniform across sparse and dense feature families, each document vector was converted into a one-dimensional feature sequence before entering the recurrent layer. This was an engineering choice for controlled comparison, not an attempt to claim an optimal neural architecture.

4 Experiments and Results

Task 1 confirmed that the corpus is large enough for embedding training but still highly sparse. Nearly half of the vocabulary occurs only once, which justifies the use of minimum frequency thresholds such as `min_count=3`.

Tasks 2 and 3 showed a consistent qualitative pattern: Word2Vec returned more coherent neighbors than GloVe, especially for political and institutional vocabulary. GloVe still captured several meaningful relations, but it produced noisier outputs on this dataset.

Task 4 reinforced this difference quantitatively. The average Jaccard overlap of 0.0111 shows that the two embedding families retrieve substantially different neighbors for the same query words.

The main classification results are reported in Table 5.

Table 5: Task 5 classification results on the `subcategory` setting

Feature	**Model**	**Test Accuracy**	**Test Macro F1**
Count	Logistic Regression baseline	0.8226	0.7211
Count	RNN	0.2460	0.1779
Count	BiRNN	0.2056	0.1591
Count	LSTM	0.2137	0.1164
TF-IDF	Logistic Regression baseline	**0.8589**	**0.7816**
TF-IDF	RNN	0.2097	0.1040
TF-IDF	BiRNN	0.2661	0.1341
TF-IDF	LSTM	0.2379	0.0963
PMI	Logistic Regression baseline	0.8387	0.7536
PMI	RNN	0.2460	0.1698
PMI	BiRNN	**0.7137**	**0.6092**
PMI	LSTM	0.2419	0.1549
Word2Vec	Logistic Regression baseline	0.7742	0.6696
Word2Vec	RNN	0.3669	0.1985
Word2Vec	BiRNN	0.5887	0.4464
Word2Vec	LSTM	0.2056	0.0672
GloVe	Logistic Regression baseline	0.6169	0.5668
GloVe	RNN	0.2782	0.1788
GloVe	BiRNN	0.2137	0.0761
GloVe	LSTM	0.3306	0.1830

The strongest overall result was TF-IDF with Logistic Regression, while the strongest assignment-required neural result was PMI with BiRNN. Word2Vec-based neural features were generally stronger than GloVe-based neural features, and LSTM was consistently the weakest recurrent model in this setup. This is an important negative result: the more complex recurrent models did not outperform the simpler linear baseline.

5 Discussion

The classification results suggest that, on this corpus, feature quality mattered more than model complexity. Sparse lexical features already separated the topic labels well, especially under TF-IDF and PMI. By contrast, the recurrent models faced three limitations. First, the label distribution was highly imbalanced, especially in the minority classes with only 50 documents. Second, document-level averaging for Word2Vec and GloVe removed word order. Third, the recurrent models operated on transformed feature sequences rather than raw token sequences.

Taken together, the experiments support three main conclusions. First, the corpus is strongly sparse and therefore requires careful frequency filtering and representation design. Second, Word2Vec is qualitatively more reliable than GloVe on this dataset. Third, for text classification on this specific corpus, TF-IDF and PMI provide stronger signals than the recurrent neural architectures tested here.

The final system also includes an interactive Streamlit UI that supports corpus inspection, embedding queries, analogy tests, and custom text classification for oral demonstration.

6 Conclusion

This project showed that the cleaned Kazakh news corpus is both informative and strongly sparse, making representation choice especially important. Word2Vec and GloVe both captured useful semantic structure, but Word2Vec produced more coherent neighbors on this dataset. In classification, the strongest results came from TF-IDF and PMI, while the neural models did not surpass the linear baseline. Overall, the experiments indicate that, for this corpus, carefully designed feature spaces were more decisive than architectural complexity.